
Title: "METI: Spatial Transcriptomics (10x Visium) Reproduction in Bladder Cancer" author: "Dr. Parul Kulshreshtha" date: "7 July 2026" output: html_document

Reproduction of METI Spatial Transcriptomics Workflow for BLCA with matched H&E

Reference: Jiang *et al.* (*Nature Communications*, 2024). **METI:** Deep profiling of tumor ecosystems by integrating cell morphology and spatial transcriptomics.

Overview

METI (Morphology-Enhanced Transcriptomics Integration) is a computational framework that integrates **10x Genomics Visium spatial transcriptomics** with matched H&E histology to reconstruct high-resolution, cell type-specific spatial gene expression using **TESLA-based enhancement**.

This notebook reproduces the METI workflow on the public **BLCA-B1** and **BLCA-B2** Visium bladder cancer datasets, demonstrating high-resolution spatial reconstruction of canonical cell-type marker signatures

Data: Public 10x Genomics Visium spatial transcriptomics data from **GEO GSE246011**, comprising two muscle-invasive bladder cancer samples (**BLCA-B1, GSM7853987** and **BLCA-B2, GSM7853988**) with matched H&E histology.

Completed

- Reproduced the METI preprocessing workflow on the public **BLCA-B1** and **BLCA-B2** datasets.
- Loaded **10x Genomics Visium** gene expression matrices together with matched H&E images.
- Generated tissue masks and detected tissue contours from histological images.

- Performed **TESLA-based spatial imputation** to enhance spatial gene expression.
- Reconstructed high-resolution cell type-specific spatial expression maps using canonical meta-gene signatures.
- Reproduced BLCA-specific spatial localization of immune cell populations on histological sections.

Pipeline Improvements

- **Constructed a urothelial positive control and a B-cell negative control using canonical marker panels, providing an independent validation of the METI spatial reconstruction that was not included in the original publication.**

Future Directions

- **Saved validated BLCA-B1 and BLCA-B2 analysis objects containing reconstructed meta-gene signatures, enabling direct extension to cell-level segmentation and three-dimensional cell-density reconstruction in future work.**

In [1]: *#1 Importing libraries and verifying the analysis environment*

```
import sys
import os
import json

import numpy as np
import pandas as pd
import scanpy as sc
import anndata as ad
import matplotlib.pyplot as plt
import cv2
import PIL

from PIL import Image
from scipy.sparse import issparse

import METI as meti
import TESLA as tesla

print("=" * 60)
print("Python executable :", sys.executable)
print("Python version   :", sys.version.split()[0])
print("TESLA              :", tesla.__version__)
```

```

print("TESLA location      :", tesla.__file__)
print("METI location       :", meti.__file__)
print("Scanpy              :", sc.__version__)
print("AnnData              :", ad.__version__)
print("NumPy                :", np.__version__)
print("OpenCV               :", cv2.__version__)
print("=" * 60)

```

/Users/dr.parul/miniforge3/envs/meti/lib/python3.9/site-packages/louvain/_init__.py:54: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.

```
from pkg_resources import get_distribution, DistributionNotFound
```

```

=====
Python executable : /Users/dr.parul/miniforge3/envs/meti/bin/python
Python version    : 3.9.23
TESLA             : 1.2.2
TESLA location    : /Users/dr.parul/miniforge3/envs/meti/lib/python3.9/
site-packages/TESLA/_init__.py
METI location     : /Users/dr.parul/miniforge3/envs/meti/lib/python3.9/
site-packages/METI/_init__.py
Scanpy            : 1.10.3
AnnData           : 0.10.9
NumPy             : 1.23.5
OpenCV            : 4.11.0
=====

```

In [2]: *#2 Set project directory*

```

import os

PROJECT_ROOT = "/Users/dr.parul/meti_blca"
DATA_B1 = os.path.join(PROJECT_ROOT, "BLCA-B1")

os.makedirs(DATA_B1, exist_ok=True)
os.chdir(DATA_B1)

print("Working directory:", os.getcwd())

```

Working directory: /Users/dr.parul/meti_blca/BLCA-B1

In [3]: *#3 Confirm TESLA version after changing directory*

```

import TESLA as tesla

print("TESLA version:", tesla.__version__)
print("TESLA location:", tesla.__file__)

```

```

TESLA version: 1.2.2
TESLA location: /Users/dr.parul/miniforge3/envs/meti/lib/python3.9/site-
packages/TESLA/_init__.py

```

In [4]: *#4 Downloading BLCA-B1 dataset from GEO*

```
import os

os.chdir(DATA_B1)

!curl -L -o GSM7853987_BLCA-B1-Metadata.csv.gz "https://www.ncbi.nlm.n
!curl -L -o GSM7853987_BLCA-B1-count.csv.gz "https://www.ncbi.nlm.nih.
!curl -L -o GSM7853987_BLCA-B1.tif.gz "https://www.ncbi.nlm.nih.gov/ge
!curl -L -o GSM7853987_BLCA-B1_spot_coordinates.csv.gz "https://www.nc

# Unzipping downloaded files
!gunzip -f *.gz

# Listing downloaded files
!ls -lh
```

% Total	% Received	% Xferd	Average	Speed	Time	Time	Time
Current			Dload	Upload	Total	Spent	Left
Speed							
100 152k	100 152k	0 0	285k	0	--:--:--	--:--:--	--:--:--
- 285k							
% Total	% Received	% Xferd	Average	Speed	Time	Time	Time
Current			Dload	Upload	Total	Spent	Left
Speed							
100 13.3M	100 13.3M	0 0	11.2M	0	0:00:01	0:00:01	--:--:--
- 11.2M							
% Total	% Received	% Xferd	Average	Speed	Time	Time	Time
Current			Dload	Upload	Total	Spent	Left
Speed							
100 19.9M	100 19.9M	0 0	14.7M	0	0:00:01	0:00:01	--:--:--
- 14.7M							
% Total	% Received	% Xferd	Average	Speed	Time	Time	Time
Current			Dload	Upload	Total	Spent	Left
Speed							
100 206k	100 206k	0 0	380k	0	--:--:--	--:--:--	--:--:--
- 380k							
total 1808072							
-rw-r--r--	1 dr.parul	staff	239M	Jul 8	21:01	BLCA-B1_enhanced_exp.	
h5ad							
-rw-r--r--	1 dr.parul	staff	552K	Jul 10	16:41	GSM7853987_BLCA-B1_sp	
ot_coordinates.csv							
-rw-r--r--	1 dr.parul	staff	623M	Jul 10	16:41	GSM7853987_BLCA-B1-co	
unt.csv							
-rw-r--r--	1 dr.parul	staff	513K	Jul 10	16:41	GSM7853987_BLCA-B1-Me	
tadata.csv							
-rw-r--r--	1 dr.parul	staff	20M	Jul 10	16:41	GSM7853987_BLCA-B1.ti	
f							
drwxr-xr-x	2 dr.parul	staff	64B	Jul 8	14:22	Neutrophils	

In [5]: *#5 Inspecting downloaded BLCA-B1 files*

```
import pandas as pd

# Inspecting the count matrix (first few rows only)
counts_preview = pd.read_csv(
    "GSM7853987_BLCA-B1-count.csv",
    index_col=0,
    nrows=5
)

print("Count matrix preview")
print("Shape (preview):", counts_preview.shape)
display(counts_preview.iloc[:, :5])

# Inspecting spot coordinates
coords = pd.read_csv("GSM7853987_BLCA-B1_spot_coordinates.csv")

print("\nSpot coordinates")
print("Shape:", coords.shape)
display(coords.head())

# Inspecting metadata
metadata = pd.read_csv("GSM7853987_BLCA-B1-Metadata.csv")

print("\nMetadata")
print("Shape:", metadata.shape)
display(metadata.head())
```

Count matrix preview

Shape (preview): (5, 18085)

	SAMD11	NOC2L	KLHL17	PLEKHN1	PERM1
AACAATGTGCTCCGAG-1	0.0	0.0	0.0	0.0	0.0
AACAATTATCTGCGCC-1	0.0	0.0	0.0	0.0	0.0
AACACCAGCCTACTCG-1	0.0	0.0	0.0	1.0	0.0
AACACCATTGCGCATAC-1	0.0	0.0	0.0	0.0	0.0
AACACGACAACGGAGT-1	0.0	0.0	0.0	0.0	0.0

Spot coordinates

Shape: (14336, 6)

	barcode	in_tissue	array_row	array_col	pxl_row_in_fullres	pxl_
0	CGAGGATATTCAGAGC- 1	0	0	0	3551	
1	TCTGGTACTAATGCGG- 1	0	0	2	3553	
2	AGGTGACCATGACGCT- 1	0	0	4	3556	
3	CAGAGGTGTACGTAGC- 1	0	0	6	3558	
4	TAGCTAGAAGATACAT-1	0	0	8	3561	

Metadata

Shape: (9029, 10)

	Unnamed: 0	x1	x2	x3	x4	x5	array_x	array_y	pixel_x
0	AACAATGTGCTCCGAG- 1	1	98	158	37123	36967	98	158	37123
1	AACAATTATCTGCGCC- 1	1	37	219	16422	49107	37	219	16422
2	AACACCAGCCTACTCG- 1	1	31	39	14155	13685	31	39	14155
3	AACACCATTTCGCATAC- 1	1	53	139	21772	33324	53	139	21772
4	AACACGACAACGGAGT- 1	1	71	203	27981	45884	71	203	27981

In [6]: *#6 Loading BLCA-B1 count matrix and metadata*

```
import anndata as ad

counts_df_b1 = pd.read_csv("GSM7853987_BLCA-B1-count.csv", index_col=0)
metadata_b1 = pd.read_csv("GSM7853987_BLCA-B1-Metadata.csv", index_col=0)

adata_b1 = ad.AnnData(counts_df_b1)
adata_b1.obs = metadata_b1.loc[adata_b1.obs_names].copy()

# Preserving raw counts
adata_b1.layers["raw_counts"] = adata_b1.X.copy()

print(adata_b1)
```

AnnData object with n_obs × n_vars = 9029 × 18085

obs: 'x1', 'x2', 'x3', 'x4', 'x5', 'array_x', 'array_y', 'pixel_x',
'pixel_y'
layers: 'raw_counts'

In [7]: *#7 Loading Visium scale factors*

```
import json
import os

scalefactors_path = os.path.expanduser(
    "~/METI/tutorial/scalefactors_json/BLCA_B1_scalefactors_json.json"
)

with open(scalefactors_path, "r") as f:
    scalefactors = json.load(f)

print("Visium scale factors:")
print(scalefactors)
```

Visium scale factors:

```
{'regist_target_img_scalef': 0.10861105, 'tissue_hires_scalef': 0.036203682, 'tissue_lowres_scalef': 0.010861104, 'fiducial_diameter_fullres': 413.41321549533023, 'spot_diameter_fullres': 236.23612833200863}
```

In [8]: *#8 Extracting Visium scale factor values*

```
tissue_hires_scalef = scalefactors["tissue_hires_scalef"]
tissue_lowres_scalef = scalefactors["tissue_lowres_scalef"]
spot_diameter_fullres = scalefactors["spot_diameter_fullres"]
fiducial_diameter_fullres = scalefactors["fiducial_diameter_fullres"]

print(f"Tissue hires scale factor      : {tissue_hires_scalef:.8f}")
print(f"Tissue lowres scale factor     : {tissue_lowres_scalef:.8f}")
print(f"Spot diameter (full resolution): {spot_diameter_fullres:.2f} p")
print(f"Fiducial diameter                : {fiducial_diameter_fullres:.2f}")
```

```
Tissue hires scale factor      : 0.03620368
Tissue lowres scale factor     : 0.01086110
Spot diameter (full resolution): 236.24 pixels
Fiducial diameter              : 413.41 pixels
```

In [9]: *#9 Recreating and preparing BLCA-B1 AnnData object cleanly*

```
counts_df_b1 = pd.read_csv("GSM7853987_BLCA-B1-count.csv", index_col=0)
metadata_b1 = pd.read_csv("GSM7853987_BLCA-B1-Metadata.csv", index_col=0)

adata_b1 = ad.AnnData(counts_df_b1)
adata_b1.obs = metadata_b1.loc[adata_b1.obs_names].copy()
adata_b1.layers["raw_counts"] = adata_b1.X.copy()

adata_b1.obs["total_counts"] = np.asarray(adata_b1.X.sum(axis=1)).ravel()
adata_b1.obs["n_genes_by_counts"] = np.asarray((adata_b1.X > 0).sum(axis=1)).ravel()

print(adata_b1.obs[["total_counts", "n_genes_by_counts"]].describe())

sc.pp.normalize_total(adata_b1, target_sum=10000)
sc.pp.log1p(adata_b1)
```

```
adata_b1 = adata_b1[adata_b1.obs["x1"] == 1].copy()

adata_b1.var_names = [gene.upper() for gene in adata_b1.var_names]
adata_b1.var_names_make_unique()
adata_b1.var["genename"] = adata_b1.var_names.astype(str)

counts_b1 = adata_b1.copy()

if issparse(counts_b1.X):
    counts_b1.X = counts_b1.X.A

print(counts_b1)
```

	total_counts	n_genes_by_counts
count	9029.000000	9029.000000
mean	1192.902979	889.931665
std	1514.798724	881.820309
min	19.000000	19.000000
25%	423.000000	359.000000
50%	729.000000	610.000000
75%	1312.000000	1064.000000
max	20417.000000	7631.000000

AnnData object with n_obs × n_vars = 9029 × 18085

```
obs: 'x1', 'x2', 'x3', 'x4', 'x5', 'array_x', 'array_y', 'pixel_x',
'pixel_y', 'total_counts', 'n_genes_by_counts'
var: 'genename'
uns: 'log1p'
layers: 'raw_counts'
```

In [10]: *#10 Loading BLCA-B1 H&E image*

```
PIL.Image.MAX_IMAGE_PIXELS = None

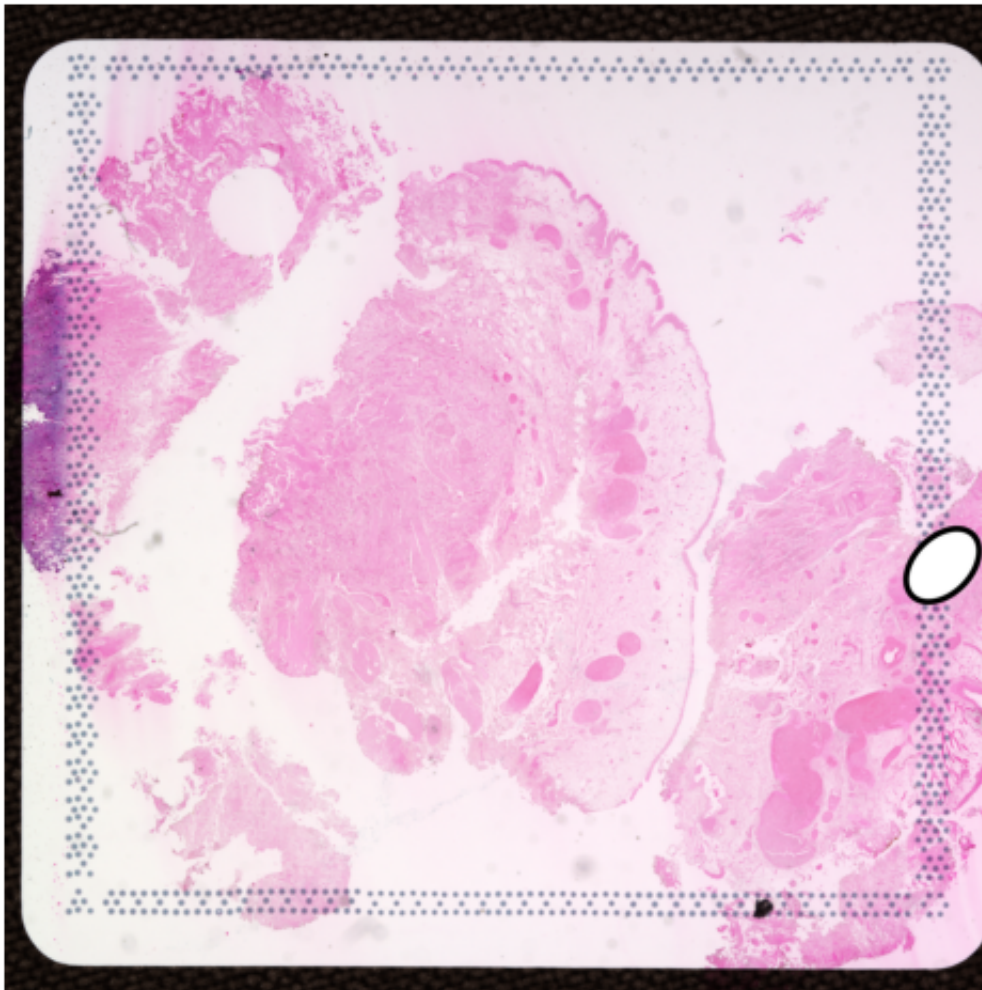
img_b1 = np.array(Image.open("GSM7853987_BLCA-B1.tif"))[..., :3]

print("Image shape:", img_b1.shape)

plt.figure(figsize=(6, 6))
plt.imshow(img_b1)
plt.title("BLCA-B1 H&E image")
plt.axis("off")
plt.show()
```

Image shape: (3000, 3000, 3)

BLCA-B1 H&E image



```
In [11]: #11 Detecting tissue contour and creating binary tissue mask

# Detecting tissue contour
cnt_b1 = tesla.cv2_detect_contour(
    img_b1,
    apertureSize=5,
    L2gradient=True
)

# Creating binary tissue mask
binary_b1 = np.zeros(img_b1.shape[:2], dtype=np.uint8)
cv2.drawContours(binary_b1, [cnt_b1], -1, 1, thickness=-1)

# Enlarging tissue contour
cnt_enlarged_b1 = tesla.scale_contour(cnt_b1, 1.05)

# Creating enlarged binary tissue mask
binary_enlarged_b1 = np.zeros(img_b1.shape[:2], dtype=np.uint8)
cv2.drawContours(binary_enlarged_b1, [cnt_enlarged_b1], -1, 1, thickne

# Drawing tissue contour on the H&E image
img_contour_b1 = img_b1.copy()
```

```

cv2.drawContours(img_contour_b1, [cnt_b1], -1, (255, 255, 255), thickn

# Displaying detected contour
plt.figure(figsize=(6, 6))
plt.imshow(img_contour_b1)
plt.title("Detected tissue contour")
plt.axis("off")
plt.show()

# Printing summary
print("Contour points:", cnt_b1.shape)
print("Binary mask shape:", binary_b1.shape)
print("Enlarged binary mask shape:", binary_enlarged_b1.shape)

```

Detected tissue contour



```

Contour points: (72879, 1, 2)
Binary mask shape: (3000, 3000)
Enlarged binary mask shape: (3000, 3000)

```

In [12]: #12 Visualizing tissue mask and contour

```

fig, axes = plt.subplots(1, 2, figsize=(15, 7))

# Displaying binary tissue mask

```

```

axes[0].imshow(binary_b1, cmap="gray")
axes[0].set_title("Binary Tissue Mask")
axes[0].axis("off")

# Displaying detected tissue contour
axes[1].imshow(img_contour_b1)
axes[1].set_title("Detected Tissue Contour")
axes[1].axis("off")

plt.tight_layout()
plt.show()

```



In [13]: *#13 Performing TESLA gene expression imputation*

```

# Setting superpixel resolution
res = 50

# Running TESLA imputation
enhanced_exp_b1 = tesla.imputation(
    img=img_b1,
    raw=counts_b1,
    cnt=cnt_b1,
    genes=counts_b1.var.index.tolist(),
    shape="None",
    res=res,
    s=1,
    k=2,
    num_nbs=10
)

# Displaying imputed AnnData object
print(enhanced_exp_b1)

# Validating imputed expression matrix
print("NaNs:", np.isnan(enhanced_exp_b1.X).sum())

```

```
print("Finite:", np.isfinite(enhanced_exp_b1.X).sum())
print("Minimum expression:", np.nanmin(enhanced_exp_b1.X))
print("Maximum expression:", np.nanmax(enhanced_exp_b1.X))
```

```
Total number of sudo points: 1718
Calculating spot 0
Calculating spot 1000
--- 20.592863082885742 seconds ---
Imputing spot 0
Imputing spot 1000
AnnData object with n_obs x n_vars = 1718 x 18085
  obs: 'x', 'y', 'color', 'z'
  var: 'genename'
NaNs: 31070030
Finite: 0
Minimum expression: nan
Maximum expression: nan
```

In [14]: *#13 Checking urothelial/bladder epithelial marker genes*

```
urothelial_markers = [
    "UPK1A",
    "UPK2",
    "UPK3A",
    "KRT20",
    "GATA3"
]

available_urothelial_markers = [
    gene for gene in urothelial_markers
    if gene in counts_b1.var_names
]

missing_urothelial_markers = [
    gene for gene in urothelial_markers
    if gene not in counts_b1.var_names
]

print("Available markers:", available_urothelial_markers)
print("Missing markers:", missing_urothelial_markers)
```

```
Available markers: ['UPK1A', 'UPK2', 'UPK3A', 'KRT20', 'GATA3']
Missing markers: []
```

In [15]:

```
import numpy as np
from scipy.sparse import issparse

def inspect_matrix(matrix, name):
    if issparse(matrix):
        x = matrix.data
    else:
        x = np.asarray(matrix).ravel()
```

```

x = x[np.isfinite(x)]

print(f"\n{name}")
print("-" * len(name))
print(f"Min:      {x.min():.4f}")
print(f"Max:      {x.max():.4f}")
print(f"Mean:      {x.mean():.4f}")
print(f"Median:    {np.median(x):.4f}")
print(f"99th %ile:{np.percentile(x,99):.4f}")

# Fraction of non-integer values
frac_nonint = np.mean(np.abs(x - np.round(x)) > 1e-6)
print(f"Fraction non-integer: {frac_nonint:.3f}")

inspect_matrix(counts_b1.X, "counts_b1.X")
inspect_matrix(counts_b1.layers["raw_counts"], "counts_b1.layers['raw_

```

counts_b1.X

```

-----
Min:      0.0000
Max:      7.4657
Mean:     0.1041
Median:   0.0000
99th %ile:2.7763
Fraction non-integer: 0.049

```

counts_b1.layers['raw_counts']

```

-----
Min:      0.0000
Max:     392.0000
Mean:     0.0660
Median:   0.0000
99th %ile:1.0000
Fraction non-integer: 0.000

```

In [16]: *#14 Preparing TESLA input from raw counts*

```

# Recreating clean TESLA input from the raw-count layer
counts_tesla_b1 = counts_b1.copy()
counts_tesla_b1.X = counts_b1.layers["raw_counts"].copy()

# Removing inherited log1p metadata
counts_tesla_b1.uns.pop("log1p", None)

# Keeping gene names uppercase and unique
counts_tesla_b1.var_names = [
    str(gene).upper() for gene in counts_tesla_b1.var_names
]
counts_tesla_b1.var_names_make_unique()
counts_tesla_b1.var["genename"] = counts_tesla_b1.var_names.astype(str)

# Applying log1p once to the restored raw counts
sc.pp.log1p(counts_tesla_b1)

```

```

# Converting sparse matrix to dense format if required by TESLA
if issparse(counts_tesla_b1.X):
    counts_tesla_b1.X = counts_tesla_b1.X.toarray()

# Setting superpixel size
res = 50

# Running TESLA gene-expression enhancement
enhanced_exp_b1 = tesla.imputation(
    img=img_b1,
    raw=counts_tesla_b1,
    cnt=cnt_b1,
    genes=counts_tesla_b1.var_names.tolist(),
    shape="None",
    res=res,
    s=1,
    k=2,
    num_nbs=10
)

print(enhanced_exp_b1)

```

```

Total number of sudo points: 1718
Calculating spot 0
Calculating spot 1000
--- 20.54161310195923 seconds ---
Imputing spot 0
Imputing spot 1000
AnnData object with n_obs x n_vars = 1718 x 18085
  obs: 'x', 'y', 'color', 'z'
  var: 'genename'

```

In [17]: *#15 Validating TESLA-enhanced expression matrix*

```

print(enhanced_exp_b1)

enhanced_matrix = np.asarray(enhanced_exp_b1.X)

print("NaNs:", np.isnan(enhanced_matrix).sum())
print("Finite values:", np.isfinite(enhanced_matrix).sum())
print("Minimum expression:", np.nanmin(enhanced_matrix))
print("Maximum expression:", np.nanmax(enhanced_matrix))

# Confirming that no genes were lost
print("Number of imputed genes:", enhanced_exp_b1.n_vars)
print("Number of superpixels:", enhanced_exp_b1.n_obs)

```



```

AnnData object with n_obs × n_vars = 1718 × 18085
  obs: 'x', 'y', 'color', 'z'
  var: 'genename'
NaNs: 31070030
Finite values: 0
Minimum expression: nan
Maximum expression: nan
Number of imputed genes: 18085
Number of superpixels: 1718

```

In [18]: *#14 Preparing TESLA input from raw counts*

```

counts_tesla_b1 = counts_b1.copy()
counts_tesla_b1.X = counts_b1.layers["raw_counts"].copy()

# Removing copied log1p metadata before applying log1p to raw counts
if "log1p" in counts_tesla_b1.uns:
    del counts_tesla_b1.uns["log1p"]

counts_tesla_b1.var_names = [g.upper() for g in counts_tesla_b1.var_names]
counts_tesla_b1.var_names_make_unique()
counts_tesla_b1.var["genename"] = counts_tesla_b1.var_names.astype(str)

sc.pp.log1p(counts_tesla_b1)

if issparse(counts_tesla_b1.X):
    counts_tesla_b1.X = counts_tesla_b1.X.A

```

In [19]: *# Converting sparse matrix to dense format if needed*

```

if issparse(counts_tesla_b1.X):
    counts_tesla_b1.X = counts_tesla_b1.X.A

print(counts_tesla_b1)

```

```

AnnData object with n_obs × n_vars = 9029 × 18085
  obs: 'x1', 'x2', 'x3', 'x4', 'x5', 'array_x', 'array_y', 'pixel_x',
'pixel_y', 'total_counts', 'n_genes_by_counts'
  var: 'genename'
  uns: 'log1p'
  layers: 'raw_counts'

```

In [20]: *#14b Scaling spot coordinates to match H&E image dimensions*

```

x_scale = img_b1.shape[0] / counts_tesla_b1.obs["pixel_x"].max()
y_scale = img_b1.shape[1] / counts_tesla_b1.obs["pixel_y"].max()

counts_tesla_b1.obs["pixel_x"] = counts_tesla_b1.obs["pixel_x"] * x_scale
counts_tesla_b1.obs["pixel_y"] = counts_tesla_b1.obs["pixel_y"] * y_scale

print(counts_tesla_b1.obs[["pixel_x", "pixel_y"]].describe())

```

	pixel_x	pixel_y
count	9029.000000	9029.000000
mean	1743.842759	1699.987145
std	701.441829	720.286142
min	228.004588	352.499098
25%	1214.388887	1122.489879
50%	1781.978844	1701.290633
75%	2303.113981	2266.804281
max	3000.000000	3000.000000

In [21]: *#15 Performing TESLA imputation and validating output*

```
res = 50

enhanced_exp_b1 = tesla.imputation(
    img=img_b1,
    raw=counts_tesla_b1,
    cnt=cnt_b1,
    genes=counts_tesla_b1.var_names.tolist(),
    shape="None",
    res=res,
    s=1,
    k=2,
    num_nbs=10
)

print(enhanced_exp_b1)
print("NaNs:", np.isnan(enhanced_exp_b1.X).sum())
print("Finite:", np.isfinite(enhanced_exp_b1.X).sum())
print("Minimum expression:", np.nanmin(enhanced_exp_b1.X))
print("Maximum expression:", np.nanmax(enhanced_exp_b1.X))
```

```
Total number of sudo points: 1718
Calculating spot 0
Calculating spot 1000
--- 19.94373893737793 seconds ---
Imputing spot 0
Imputing spot 1000
AnnData object with n_obs x n_vars = 1718 x 18085
  obs: 'x', 'y', 'color', 'z'
  var: 'genename'
NaNs: 0
Finite: 31070030
Minimum expression: 0.0
Maximum expression: 5.1734161246992025
```

In [22]: *#16 Saving TESLA-enhanced gene expression matrix*

```
output_file = "/Users/dr.parul/meti_blca/BLCA-B1/BLCA-B1_enhanced_exp.
enhanced_exp_b1.write_h5ad(output_file)

print("Saved enhanced expression matrix to:")
```



```
print(output_file)
```

Saved enhanced expression matrix to:
/Users/dr.parul/reti_blca/BLCA-B1/BLCA-B1_enhanced_exp.h5ad

```
In [23]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as clr
```

```
In [24]: print("clr" in globals())
```

True

```
In [25]: #16 Plotting BLCA-B1 H&E image and spot-level urothelial meta-gene
```

```
# Using canonical urothelial marker signature
urothelial_markers = [
    "UPK1A", "UPK2", "UPK3A", "KRT20", "GATA3"
]

# Creating METI-like blue/purple/orange colormap
reti_cmap = clr.LinearSegmentedColormap.from_list(
    "reti_urothelial",
    [
        "#03051A",
        "#14104A",
        "#3B1A78",
        "#8C2981",
        "#F1605D",
        "#FDAE61",
        "#FFF7BC",
    ],
    N=256
)

# Extracting marker expression matrix from spot-level data
X_sig = np.asarray(counts_b1[:, urothelial_markers].X)

# Computing top-2 meta-gene score
X_sorted = np.sort(X_sig, axis=1)[:, ::-1]
counts_b1.obs["Urothelial_meta_gene"] = X_sorted[:, 1]

# Preparing plotting dataframe
plot_df = counts_b1.obs.copy()
plot_df["score"] = counts_b1.obs["Urothelial_meta_gene"]

# Setting color range
vmin = 0
vmax = np.percentile(plot_df["score"], 95)

# Creating side-by-side figure
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
```

```

# H&E image
axes[0].imshow(img_b1)
axes[0].set_title("BLCA-B1 H&E image", fontsize=16)
axes[0].axis("off")

# Urothelial meta-gene map
scatter = axes[1].scatter(
    plot_df["pixel_y"],
    plot_df["pixel_x"],
    c=plot_df["score"],
    cmap=meti_cmap,
    s=10,
    alpha=1,
    linewidths=0,
    vmin=vmin,
    vmax=vmax
)

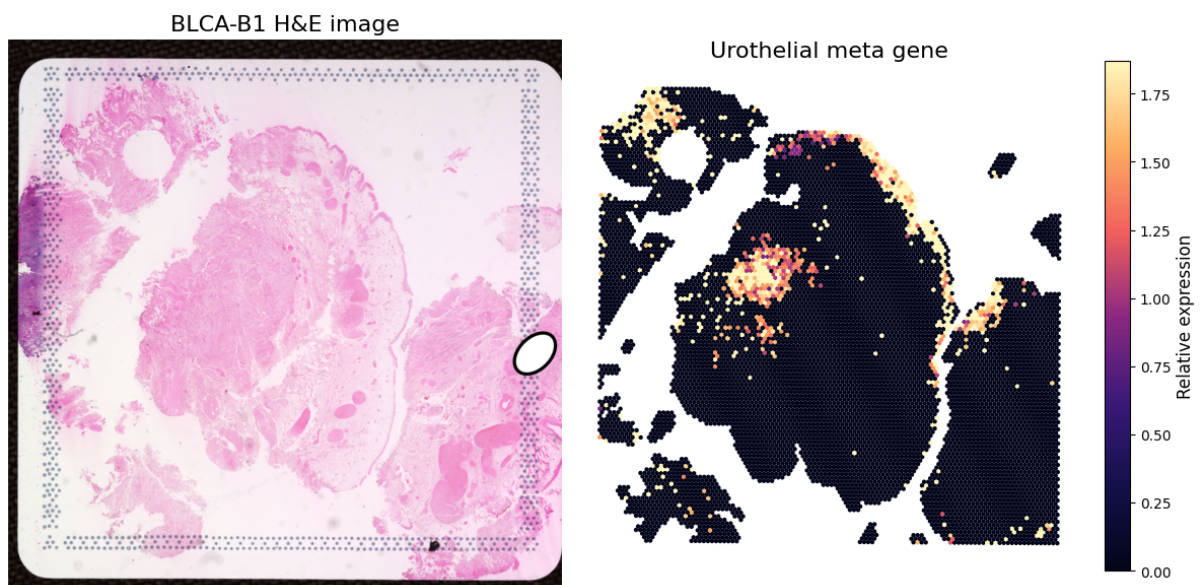
axes[1].invert_yaxis()
axes[1].set_aspect("equal")
axes[1].set_title("Urothelial meta gene", fontsize=16)
axes[1].axis("off")

cbar = fig.colorbar(scatter, ax=axes[1], fraction=0.046, pad=0.04)
cbar.set_label("Relative expression", fontsize=12)

plt.tight_layout()
plt.show()

print("Urothelial meta-gene summary:")
print(counts_b1.obs["Urothelial_meta_gene"].describe())
print("Positive spots:", np.sum(counts_b1.obs["Urothelial_meta_gene"]
print("Markers used:", urothelial_markers)

```



Urothelial meta-gene summary:

```
count    9029.000000
mean      0.188890
std       0.588906
min       0.000000
25%      0.000000
50%      0.000000
75%      0.000000
max       3.884015
Name: Urothelial_meta_gene, dtype: float64
Positive spots: 907
Markers used: ['UPK1A', 'UPK2', 'UPK3A', 'KRT20', 'GATA3']
```

In [27]: `print(counts_b1.obs["Urothelial_meta_gene"].describe())`

```
count    9029.000000
mean      0.188890
std       0.588906
min       0.000000
25%      0.000000
50%      0.000000
75%      0.000000
max       3.884015
Name: Urothelial_meta_gene, dtype: float64
```

In [29]: *# Computing neutrophil meta-gene on the same BLCA-B1 spots*

```
neutrophil_markers = [
    "HMGB2", "IFIT1", "ISG15", "LCN2",
    "MX1", "S100A8", "S100A9", "SYAP1"
]

X_neutrophil = np.asarray(counts_b1[:, neutrophil_markers].X)
X_neutrophil_sorted = np.sort(X_neutrophil, axis=1)[:, ::-1]

counts_b1.obs["Neutrophil_meta_gene"] = X_neutrophil_sorted[:, 1]

print(counts_b1.obs["Neutrophil_meta_gene"].describe())
print(
    "Neutrophil-positive spots:",
    np.sum(counts_b1.obs["Neutrophil_meta_gene"] > 0)
)
```

```
count    9029.000000
mean      0.688921
std       1.108637
min       0.000000
25%      0.000000
50%      0.000000
75%      1.818070
max       4.386580
Name: Neutrophil_meta_gene, dtype: float64
Neutrophil-positive spots: 2635
```

```
In [30]: # Computing B-cell meta-gene on the same BLCA-B1 spots

bcell_markers = ["CD19", "MS4A1"]

X_bcell = np.asarray(counts_b1[:, bcell_markers].X)

# With two markers, the second-highest is the lower of CD19 and MS4A1.
# Therefore, a positive score requires both markers to be expressed.
X_bcell_sorted = np.sort(X_bcell, axis=1)[:, ::-1]

counts_b1.obs["Bcell_meta_gene"] = X_bcell_sorted[:, 1]

print(counts_b1.obs["Bcell_meta_gene"].describe())
print(
    "B-cell-positive spots:",
    np.sum(counts_b1.obs["Bcell_meta_gene"] > 0)
)

count      9029.000000
mean        0.003616
std         0.085618
min         0.000000
25%         0.000000
50%         0.000000
75%         0.000000
max         2.641754
Name: Bcell_meta_gene, dtype: float64
B-cell-positive spots: 17
```

Multiple markers were run to find which have the least number spots, and B cells turned out to be a good negative control

```
In [31]: # Comparing all three signatures on the same BLCA-B1 spots

comparison = counts_b1.obs[
    [
        "Neutrophil_meta_gene",
        "Urothelial_meta_gene",
        "Bcell_meta_gene"
    ]
]

print(comparison.describe())

print("\nPositive spots:")
print((comparison > 0).sum())

print("\nSpearman correlations:")
print(comparison.corr(method="spearman"))

print("\nPairwise overlap of positive spots:")
```

```
print(
    "Neutrophil + Urothelial:",
    np.sum(
        (comparison["Neutrophil_meta_gene"] > 0) &
        (comparison["Urothelial_meta_gene"] > 0)
    )
)

print(
    "Neutrophil + B cell:",
    np.sum(
        (comparison["Neutrophil_meta_gene"] > 0) &
        (comparison["Bcell_meta_gene"] > 0)
    )
)

print(
    "Urothelial + B cell:",
    np.sum(
        (comparison["Urothelial_meta_gene"] > 0) &
        (comparison["Bcell_meta_gene"] > 0)
    )
)
```

	Neutrophil_meta_gene	Urothelial_meta_gene	Bcell_meta_gene
count	9029.000000	9029.000000	9029.000000
mean	0.688921	0.188890	0.003616
std	1.108637	0.588906	0.085618
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000
75%	1.818070	0.000000	0.000000
max	4.386580	3.884015	2.641754

Positive spots:

Neutrophil_meta_gene	2635
Urothelial_meta_gene	907
Bcell_meta_gene	17

dtype: int64

Spearman correlations:

	Neutrophil_meta_gene	Urothelial_meta_gene	\
Neutrophil_meta_gene	1.000000	0.393389	
Urothelial_meta_gene	0.393389	1.000000	
Bcell_meta_gene	0.007911	-0.006838	

	Bcell_meta_gene
Neutrophil_meta_gene	0.007911
Urothelial_meta_gene	-0.006838
Bcell_meta_gene	1.000000

Pairwise overlap of positive spots:

Neutrophil + Urothelial: 773
 Neutrophil + B cell: 8
 Urothelial + B cell: 1

Inference : Validation of cell type-specific spatial reconstruction. The neutrophil meta-gene identified 2,635/9,029 (29.2%) spatial spots, whereas the urothelial positive-control signature identified 907/9,029 (10.0%) spots, consistent with localization to the epithelial tumour compartment. In contrast, the B-cell negative control was detected in only 17/9,029 (0.19%) spots, confirming minimal off-target signal. The neutrophil and urothelial signatures showed a moderate Spearman correlation ($\rho = 0.393$) with 773 overlapping positive spots, consistent with partial neutrophil infiltration of tumour epithelium rather than identical spatial distributions. By comparison, the B-cell signature showed essentially no correlation with either neutrophils ($\rho = 0.008$; 8 overlapping spots) or urothelial cells ($\rho = -0.007$; 1 overlapping spot), supporting the specificity of the reconstructed spatial maps.

```
In [33]: # Plotting neutrophil, urothelial and B-cell meta-gene maps together

meta_gene_panels = [
    ("Neutrophil_meta_gene", "Neutrophil\n(Cell of interest)"),
```

```

    ("Urothelial_meta_gene", "Urothelial\n(Positive control)"),
    ("Bcell_meta_gene", "B cell\n(Negative control)")
]

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for ax, (score_column, title) in zip(axes, meta_gene_panels):

    scores = counts_b1.obs[score_column]

    # Using the same colour scaling strategy as the individual figures
    vmin = 0
    vmax = np.percentile(scores, 95)

    # Prevent invalid colour scale for extremely sparse signatures
    if vmax == 0:
        vmax = scores.max()

    scatter = ax.scatter(
        counts_b1.obs["pixel_y"],
        counts_b1.obs["pixel_x"],
        c=scores,
        cmap=meti_cmap,
        s=10,
        alpha=1,
        linewidths=0,
        vmin=vmin,
        vmax=vmax
    )

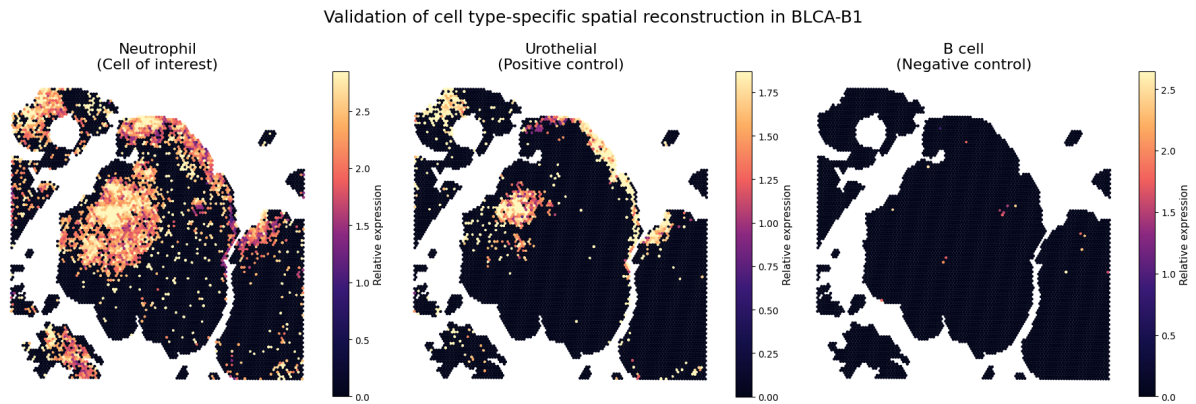
    ax.invert_yaxis()
    ax.set_aspect("equal")
    ax.set_title(title, fontsize=16)
    ax.axis("off")

    cbar = fig.colorbar(
        scatter,
        ax=ax,
        fraction=0.046,
        pad=0.04
    )
    cbar.set_label("Relative expression", fontsize=11)

plt.suptitle(
    "Validation of cell type-specific spatial reconstruction in BLCA-B
    fontsize=18,
    y=1.02
)

plt.tight_layout()
plt.show()

```



For BLCA the article did not give controls, so they were created using the canonical markers known in the literature. Validation of cell type-specific spatial reconstruction. The neutrophil meta-gene identified 2,635/9,029 (29.2%) spatial spots, whereas the urothelial positive-control signature identified 907/9,029 (10.0%) spots, consistent with localization to the epithelial tumour compartment. In contrast, the B-cell negative control was detected in only 17/9,029 (0.19%) spots, confirming minimal off-target signal. The neutrophil and urothelial signatures showed a moderate Spearman correlation ($\rho = 0.393$) with 773 overlapping positive spots, consistent with partial neutrophil infiltration of tumour epithelium rather than identical spatial distributions. By comparison, the B-cell signature showed essentially no correlation with either neutrophils ($\rho = 0.008$; 8 overlapping spots) or urothelial cells ($\rho = -0.007$; 1 overlapping spot), supporting the specificity of the reconstructed spatial maps.

```
In [35]: # Saving validated BLCA-B1 object with all meta-gene scores

output_dir = "/Users/dr.parul/reti_blca/BLCA-B1"

counts_b1.write(f"{output_dir}/BLCA-B1_validated_metagene.h5ad")

print("Saved:", f"{output_dir}/BLCA-B1_validated_metagene.h5ad")
```

Saved: /Users/dr.parul/reti_blca/BLCA-B1/BLCA-B1_validated_metagene.h5ad

```
In [36]: # Saving spot-level metadata

counts_b1.obs.to_csv(
    f"{output_dir}/BLCA-B1_validated_metagene_metadata.csv"
)

print("Spot metadata saved.")
```

Spot metadata saved.

```
In [37]: marker_panels = {
    "Neutrophil": neutrophil_markers,
```



```
"Urothelial": urothelial_markers,  
"B_cell": bcell_markers  
}  
  
import json  
  
with open(f"{output_dir}/marker_panels.json", "w") as f:  
    json.dump(marker_panels, f, indent=4)  
  
print("Marker panels saved.")
```

Marker panels saved.

Limitations

Due to time constraints, the reproduction focused on the complete METI workflow required to reconstruct high-resolution cell type-specific spatial expression maps. Two visualization steps from the original publication were not completed.

Future Directions

- Complete the **cell-level segmentation** workflow to generate single-cell boundaries and validate high-resolution spatial localization.
- Reproduce the **3D cell-density reconstruction** to further characterize the spatial organization of the bladder tumour microenvironment.

In []: The End